

STRANGER THINGS CTF

Durga Nandhini S (22pc11)

L Shambhavi (22pc17)

Total Challenges: 8

Total Points: 3600

CHALLENGE 1 - THE HAWKINS FILES

Points: 100

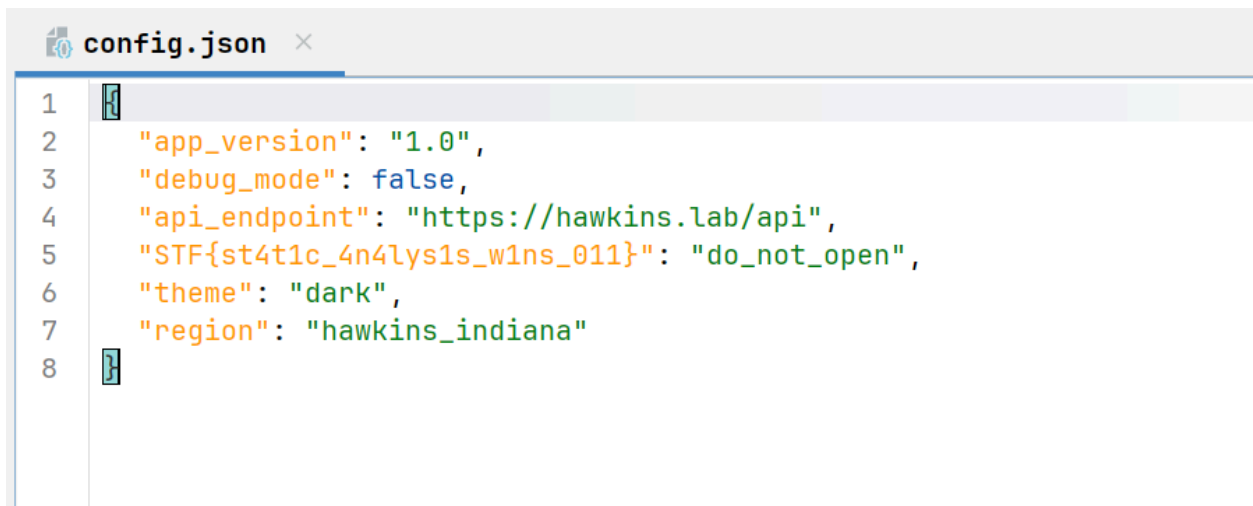
Technique: Static Analysis

The flag is stored as a JSON key in assets/config.json inside the APK.

The app checks if the player input exists as a key in that file.

Method:

Use jadx to open the apk. Under the assets folder, you will find config.json which will have the flag.



```
1 {
2   "app_version": "1.0",
3   "debug_mode": false,
4   "api_endpoint": "https://hawkins.lab/api",
5   "STF{st4t1c_4n4lys1s_w1ns_011}": "do_not_open",
6   "theme": "dark",
7   "region": "hawkins_indiana"
8 }
```

FLAG: STF{st4t1c_4n4lys1s_w1ns_011}

CHALLENGE 2 - ELEVEN'S POWERS

Points: 200

Technique: Frida Hook

The flag is returned by `getElevensPower()` at runtime.

Method 1:

Frida Script:

```
Java.perform(function() {
  var C2 = Java.use("com.example.strangerthings.challenges.Challenge2Frida");
  C2.getElevensPower.implementation = function() {
    var flag = this.getElevensPower();
    console.log("FLAG: " + flag);
    return flag;
  };
});
```

Run:

```
frida -U -f com.example.strangerthings -l challenge2.js --no-pause
```

Method 2:

This flag is kept in plain sight at the Flag Verifier function. But the player needs to find that the flag is placed in that class.

FLAG: STF{fr1d4_s33s_4ll_3l3v3n_w4tch3s}

CHALLENGE 3 - THE SHADOW GATE

Points: 300

Technique: Frida Hook + Time-based Flag

The app has 3 checks: `isRooted()`, `isDebuggable()`, `isEmulator()`.

All 3 must return false to get the real flag.

The flag changes once every two hours based on SHA-256 of a secret key + time window.

Secret key found in source via jadx: `hawkins_gate_011`

Frida Script:

```
Java.perform(function() {
  var C3 = Java.use("com.example.strangerthings.challenges.Challenge3RootDebug");
  C3.isRooted.implementation = function() { return false; };
});
```

```
C3.isDebuggable.implementation = function() { return false; };
C3.isEmulator.implementation = function() { return false; };
var flag = C3.getFlag();
console.log("FLAG: " + flag);
});
```

Run:

```
frida -U -f com.example.strangerthings -l challenge3.js --no-pause
```

If you get STF{n0t_th1s_0n3} - root check still active

If you get STF{k33p_try1ng} - debugger check still active

If you get STF{alm0st_th3r3} - emulator check still active

Method 2:

Use an alternate python code, which performs the same task as the flag generating function.

```
import hashlib, time
secret = "hawkins_gate_011"
window = int(time.time() * 1000) // 3600000
raw = f"{secret}:{window}"
hash_bytes = hashlib.sha256(raw.encode()).digest()
dynamic = hash_bytes[:8].hex()
print(f"STF{{{dynamic}_g4t3_0p3n}}")
```

FLAG: STF{<dynamic_16hex_chars>_thr33_g4t3_0p3n}

Flag changes once every two hours - hook getFlag() to get current value

CHALLENGE 4 - MIND FLAYER'S CIPHER

Points: 400

Technique: AES Decryption + SQLite Database

The flag is AES-encrypted and stored in a SQLite database on the device.

The AES key is split across the source code.

Method 1:

Step 1 - Pull the database from the device:

```
adb pull /data/data/com.example.strangerthings/databases/hawkins.db
```

Step 2 - Open hawkins.db with DB Browser for SQLite

Go to Browse Data tab, select the vault table

Find the encrypted payload for challenge 4

Or get the encrypted entry using jadx

Step 3 - Find the AES key in source code using jadx

Search for the key parts split across the code

Step 4 - Decrypt using Python:

```
from Crypto.Cipher import AES
import base64

key = b'hawkins_lab_k3y!'

encrypted = base64.b64decode('<payload from DB>')

cipher = AES.new(key, AES.MODE_ECB)

print(cipher.decrypt(encrypted).decode().strip())
```

FLAG: STF{4es_crypt0_1s_n0t_secur3}

CHALLENGE 5 - THE GATE CLOSES

Points: 500

Technique: SQLite Database

Same database as Challenge 4.

Method 1:

```
adb pull /data/data/com.example.strangerthings/databases/hawkins.db
```

Open with DB Browser for SQLite.

Or get the entry using jadx.

Go to the vault table, find id=5, decode the base64 encoded value to get the flag.

FLAG: STF{d4tab4s3_fl4g_f0und_y0u}

CHALLENGE 6 - THE LAB TAPES

Points: 600

Technique: SSL Pinning Bypass

The flag is XOR encoded and only returned when `isPinningBypassed()` is true.
By default it always returns false.

Frida Script:

```
Java.perform(function() {  
  var C6 = Java.use("com.example.strangerthings.challenges.Challenge6SSLPinning");  
  C6.isPinningBypassed.implementation = function() {  
    return true;  
  };  
  var flag = C6.getFlagIfBypassed();  
  console.log("FLAG: " + flag);  
});
```

Run:

```
frida -U -f com.example.strangerthings -l challenge6.js --no-pause
```

FLAG: STF{ssl_p1nn1ng_byp4ss3d_h4wk1ns}

CHALLENGE 7 - THE HIVE MIND

Points: 700

Technique: Heap Memory / Runtime Interception

The flag is XOR encoded and decoded into heap memory at runtime.
It is never stored as plaintext in the APK.
Hook `getSecret()` to read the value from memory.

Frida Script:

```
Java.perform(function() {  
  var C7 = Java.use("com.example.strangerthings.challenges.Challenge7HiveMind");
```

```
var inst = C7.INSTANCE;
inst.initSecret();
var flag = inst.getSecret();
console.log("FLAG: " + flag);
});
```

Run:

```
frida -U -f com.example.strangerthings -l challenge7.js --no-pause
```

FLAG: STF{h34p_m3m0ry_fl4y3r_f0und}

CHALLENGE 8 - THE UPSIDE DOWN MIRROR

Points: 800

Technique: APK Signature Verification Bypass

The app computes MD5 of its own APK signature and compares it to a hardcoded hash.

If they don't match, a decoy is returned.

Hook `isSignatureValid()` to return true and get the real flag.

Frida Script:

```
Java.perform(function() {
  var C8 = Java.use("com.example.strangerthings.challenges.Challenge8Signature");
  C8.isSignatureValid.implementation = function(context) {
    return true;
  };
  C8.getActualSigHash.implementation = function(context) {
    return "a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4";
  };
  console.log("Hooks active. Open Challenge 8 in the app.");
});
```

Run:

```
frida -U -f com.example.strangerthings -l challenge8.js --no-pause
```

FLAG: STF{s1gn4tur3_v3r1f13d}

Method 2:

For challenge 6, 7, and 8 the flag to be verified with, needs a python script which performs an XOR with different keys to decipher the flag. So, the flags can be found using the python function as well.

FLAGS:

- Challenge 1 - STF{st4t1c_4n4lys1s_w1ns_011}
- Challenge 2 - STF{fr1d4_s33s_4ll_3l3v3n_w4tch3s}
- Challenge 3 - STF{<dynamic_16hex_chars>_g4t3_0p3n}
- Challenge 4 - STF{4es_crypt0_1s_n0t_secur3}
- Challenge 5 - STF{d4tab4s3_fl4g_f0und_y0u}
- Challenge 6 - STF{ssl_p1nn1ng_byp4ss3d_h4wk1ns}
- Challenge 7 - STF{h34p_m3m0ry_fl4y3r_f0und}
- Challenge 8 - STF{s1gn4tur3_v3r1f13d}